
Assignment 7**Learning goals:**

Apply the concepts and methods presented in chapter “Symbol Table” of the lecture to some exercise problems.

Presentation:

Please prepare the solutions for informal presentation in class, and fill in the online check list in ZEUS.

Task 1 Symbol table extension

Consider the following program using an extension of MicroJava that supports the additional data type *float* for floating-point numbers.

```
program Circle
  final float PI = 3.1416;
{
  float circumference(float diameter)
  {
    return diameter * PI;
  }

  void main()
  {
    float diameter;
    {
      read(diameter);
      print(circumference(diameter));
    }
  }
}
```

Describe necessary changes to the symbol table implementation in package *MJ.SymTab* to support this MicroJava extension. Show the structure of the symbol table after parsing the opening brace of method *main()*. Implementation is *not* necessary for this task – but see Task 4.

Task 2 Type checking

Given the MicroJava language extension of Task 1, design changes in *MJ.SymTab* and *MJ.Parser* methods that check compatibility of data types in expressions, assignments, and method calls, allowing integer or floating-point operands, but not mixed expressions. Implementation is *not* required for this task.

Example program:

```
program Task2
    final int K = 5;
    int m, n;
    float p, q;
    final float R = 3.25;
{
    float product(float factor)
    {
        return factor * R;
    }

    void main()
    {
        m = 13 - 2 * K;           // OK, result 3
        p = (R + 1.0) * 4.0; // OK, result 17.0
        n = 2 * m + p;           // Error "incompatible types in expression"
        q = m - K;               // Error "incompatible types in assignment"
        print(product(K)); // Error "parameter type mismatch"
    }
}
```

Task 3 Type conversion

Similar to Task 2, but allow automatic conversion from integer to floating-point operands (but not vice-versa).

Example program:

```
program Task3
    final int K = 5;
    int m, n;
    float p, q;
    final float R = 3.25;
{
    float product(float factor)
    {
        return factor * R;
    }

    int address(int offset)
    {
        return K + offset;
    }
}
```

```

void main()
{
    m = 13 - 2 * K;           // OK, result 3
    p = (R + 1) * 4;         // OK, result 17.0
    n = 2 * m + p;           // Error "incompatible types in assignment"
    q = m - K;               // OK, -2.0 assigned to q
    print(product(K));        // OK, 5.0 is passed to method
    print(address(p));        // Error "parameter type mismatch"
}
}

```

Task 4 Floating-point extension of MJ compiler

Implement the extensions of Tasks 1 and 3 in the MicroJava compiler, such that the given code examples are parsed and checked for type compatibility. If necessary, comment out code related to code generation to avoid compiler errors.